

Skip List – Some Important Information

```
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <ctime>

using namespace std;

// Maximum level for this skip list.
// Level 3 means nodes can exist on levels 0, 1, 2, and 3.
#define MAX_LEVEL 3
```

Program Analysis

< iostream >

– It Handles standard input and output streams in C ++.

–In our code: It gives us access to cin and cout.

We need this to print the interactive menu to the screen and read the user's keyboard input .

(like choosing option "1" to insert, or typing a number to search for).

< cstdlib > (C Standard Library)

–Contains general – purpose functions, including dynamic memory management and random number generation.

–In our code: This is a heavy lifter for this specific program.

It provides malloc() and free() to create and destroy the nodes in memory (which was one of your original requirements).

It also provides rand() and srand(), which we use to flip the coin and generate the random levels for the express lanes.

< cstring > (C String)

–Provides functions for manipulating arrays of characters and blocks of memory.

–In our code: It gives us the memset() function.

When we create the update array (the "breadcrumbs" used during insertion and deletion), we need to ensure all

the pointers inside it start as completely empty (NULL).

memset lets us wipe that entire array clean in one fast line of code instead of writing a for loop.

< ctime > (C Time)

–Provides functions for fetching and manipulating the current date and time.

–It gives us time(0), which grabs the current system time.

In main(), we pass this time to srand(). Because the time is always changing, this seeds the random number generator with a unique starting point every time you run the program. If we didn't include this, rand() would generate the exact same sequence of coin flips every single time you launched the app, destroying the randomness of the Skip List.

RAND() or RANDOM

How it works: Computers cannot generate true randomness.

Instead, rand() uses a complex mathematical formula. It takes a starting number, applies a bunch of math to it, and spits out a new number. When you call it again, it takes that new number, applies the same math, and spits out the next one.

The Problem: Because it is just a fixed mathematical formula, if the starting number is always the same, the sequence of random numbers will be exactly the same every time we run the program.

By default, C++ always starts with a starting number of 1.

The Formula of RAND()

The standard rand() function in most C/C++ compilers uses an algorithm called a Linear Congruential Generator (LCG).

$$X_{n+1} = (a \cdot X_n + c)(\text{mod } m)$$

X_{n+1} → The new random number being generated.

a → The multiplier (a massive, carefully chosen constant).

X_n → The previous random number. (If this is the very first time rand() is called, this is the seed value you set with srand()).

c → The increment (another carefully chosen constant).

m → The modulus (the maximum possible limit, usually tied to the memory limit of an integer, like 2^{31}).

Lets say:

$$a = 1103515245$$

$$c = 12345$$

$$m = 2^{31} \text{ (which is 2,147,483,648).}$$

And X_n by default 1 at starting .

$$X_{n+1} = (1103515245 \cdot 1 + 12345)(\text{mod } 2^{31})$$

$$X_{n+1} = (1103527590)(\text{mod } 2147483648)$$

As, $25 \text{ mod } 10 = 5$ (remainder) [$25 > 10$] but when,

$$1 \text{ mod } 10 = 1 [1 < 10] [\text{Remainder}]$$

$$2 \text{ mod } 10 = 2 [2 < 10] [\text{Remainder}]$$

$$3 \text{ mod } 10 = 3 [3 < 10] [\text{Remainder}]$$

.

.

$$10 \text{ mod } 10 = 0 [10 = 10] [\text{Remainder}]$$

$$\therefore 1103527590 < 2147483648$$

$$\text{Hence, } X_{n+1} = 1103527590$$

Next call to rand() will become:

$$\text{Now, } X_n = 1103527590$$

$$X_{n+1} = (1103515245 \cdot 1103527590 + 12345)(\text{mod } 2^{31})$$

$$X_{n+1} = (1103515245 \cdot 1103527590 + 12345)(\text{mod } 2147483648)$$

$$1103515245 \times 1103527590 = 1,217,759,518,843,109,550$$

$$1,217,759,518,843,109,550 + 12345 = 1,217,759,518,843,121,895$$

$$1,217,759,518,843,121,895 \text{ mod } 2,147,483,648, \text{ and:}$$

$$2,147,483,648 < 1,217,759,518,843,121,895$$

$$\therefore 1,217,759,518,843,121,895 \text{ mod } 2,147,483,648 \\ = 377401575 [\text{Remainder}]$$

$$\therefore X_{n+1} = 377401575$$

TIME(0)

It fetches the current system time.

Under the hood, computers track time by counting the exact number of seconds that have passed since January 1, 1970 (known as the Unix Epoch).

In the late 1960s, a team of legendary programmers at Bell Labs (most notably Ken Thompson and Dennis Ritchie) were building a brand – new operating system called Unix. (Unix is the grandfather of Linux, macOS, Android, and iOS).

They needed a way for Unix to track time. Initially, in early drafts of the system, they actually set the zero point to January 1, 1970.

If we run time(0) to ask your C++ program what time it is, it will calculate: "How many seconds have passed since January 1, 1970, up until this exact moment today at random time say 8:52 PM?"

- ***When we call time(0), it doesn't give you 8:52 PM. It gives us a massive integer, like 1817046727.***
- ***If we call it one second later, it gives you 1817046728.***

(Note: The 0 or NULL just tells the function to return the value directly rather than trying to store it in a separate variable).

So, time(0) is always measuring the distance between January 1, 1970 and Right Now, no matter what time of day it currently is!

SRAND() stands for SEED RAND() or SEED RANDOM.

It allows you to manually set the starting number (the "seed") that rand() uses for its mathematical formula.

$$SRAND(TIME(0)):$$

It means "Take the exact number of seconds since 1970 and use THAT as the starting point for the random math."

Suppose current date and time is : 01:11:00 A.M of 31 – 07 – 2026
And it will be calculated from : January 1, 1970

Counting Years : $2025 - 1970 + 1 = 56$ years

Days without leap year : $56 \times 365 = 20440$

$$\text{Leap years : } (1972, 1976, 1980, 1984, 1988, 1992, 1996, 2000, 2004, 2008, 2012, 2016, 2020, 2024)$$

= 14 leap years

$$\therefore 20440 + 14 = 20454$$

Step 3: Count days in 2026 before July 30

Since 2026 is not a leap year,

Month	Days
January	31
February	28
March	31
April	30
May	31
June	30
July (before the 30th)	29

Now add them:

$$31 + 28 = 59$$

$$59 + 31 = 90$$

$$90 + 30 = 120$$

$$120 + 31 = 151$$

$$151 + 30 = 181$$

$$181 + 29 = 210$$

Therefore,

210 days

have elapsed before 30 July 00:00 UTC.

Step 4: Total days

$$20454 + 210 = 20664$$

So,

20664 days have elapsed.

Step 5: Convert days into seconds

One day contains

$$24 \text{ hrs} \times 60 \text{ minute} \times 60 \text{ seconds} = 86400 \text{ seconds.}$$

$$\text{Therefore, } 20664 \times 86400 = 1,785,369,600 \text{ seconds.}$$

Step 6: Convert the current time to UTC

Local time (IST):

01:11:00

IST is 5 hours 30 minutes ahead of UTC.

Subtract 5 hours 30 minutes:

$$01:11 - 05:30 = 19:41 \text{ on the previous day.}$$

So UTC time is:

19:41:00

Step 7: Convert 19:41:00 to seconds

Hours: 19 hrs $\times$ 60 minutes $\times$ 60 seconds = 68,400seconds

Minutes: 41minutes $\times$ 60 seconds = 2,460 seconds

Seconds: 0

Total: 68,400 + 2,460 + 0 = 70,860 seconds

Step 8: Final addition

Base seconds: 1,785,369,600

Current day's elapsed seconds: 70,860

Adding them: 1,785,369,600 + 70,860 = 1,785,440,460

If we run it just one single second later at 01:11:01 AM, the seed becomes 1785440461.

Therefore for Random Equation:

$$X_{n+1} = (a \cdot X_n + c)(\text{mod } m)$$

Here are the constants used by some of the most popular compilers in the world:

- ***Linux (GNU C Library / glibc):***
 - ***a = 1103515245***
 - ***c = 12345***

- ***Windows (Microsoft Visual Studio):***
 - ***a = 214013***
 - ***c = 2531011***

- ***Older Systems (Borland C/C++):***
 - ***a = 22695477***
 - ***c = 1***

If we go with ,Linux (GNU C Library / glibc), we get:

$$X_{n+1} = (1103515245 \times 1,785,440,460 + 12345)(\text{mod } 2^{31})$$

memset(): The Memory Wiper

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value.

In our Skip List code, it looks like this:

```
struct Node *update[MAX_LEVEL + 1];
```

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

And

```
#define MAX_LEVEL 3
```

Then levels are:

Level → 0

Level → 1

Level → 2

Level → 3

hence, MAX_LEVEL + 1

*sizeof(struct Node *) → 8 bytes for 64 bit system architecture.*

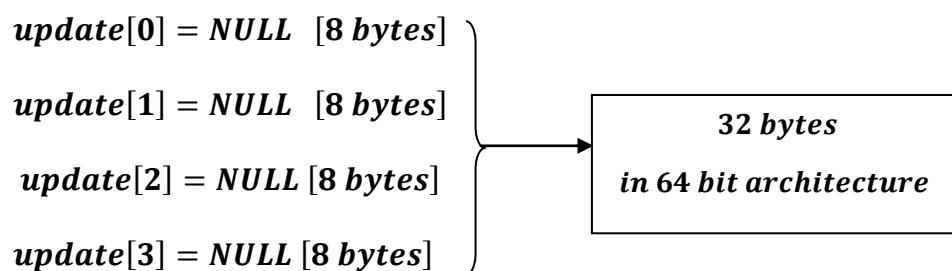
*sizeof(struct Node *) → 4 bytes for 32 bit system architecture.*

*if we take 64 bit , it will be : 8 bytes * (3 + 1) = 8 × 4 = 32 bytes.*

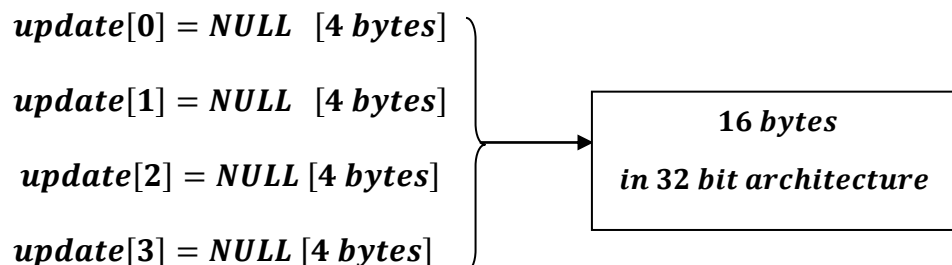
*if we take 32 bit , it will be : 4 bytes * (3 + 1) = 4 × 4 = 16 bytes.*

The Value: 0 (In C ++, setting a pointer to 0 is the exact same as setting it to NULL).

That is it will set:



[OR]



Why use it?

When we declare an array in C ++ like `struct Node * update[4];`, the computer just grabs raw memory for it. That memory might contain leftover garbage data from other programs that were recently closed. If we accidentally try to follow a garbage pointer, our program crashes.

We could clean it out using a loop:

```
for (int i = 0; i <= MAX_LEVEL; i++)  
{  
    update[i] = NULL;  
}
```

However, loops process data one item at a time. `memset()` is a highly optimized, low – level instruction that communicates directly with the processor to wipe the entire block of memory in one massive swoop. It is the fastest, safest way to ensure your update array is completely blank before we start tracking our Skip List path.
